

inside the class's methods access or modify the attributes of objects of that class. But in Python, this enforcement doesn't exist. All attributes and methods are effectively *public access*: code outside of the class can access and modify any attribute in any object of that class.

But private access is useful. For example, objects of a `BankAccount` class could have a `balance` attribute that only methods of the `BankAccount` class should have access to. For those reasons, Python's convention is to start *private* attribute or method names with a single underscore. Technically, there is nothing to stop code outside the class from accessing private attributes and methods, but it's a best practice to let only the class's methods access them.

Open a new file editor window, enter the following code, and save it as *privateExample.py*. In it, objects of a `BankAccount` class have private `_name` and `_balance` attributes that only the `deposit()` and `withdraw()` methods should directly access:

```
class BankAccount:
    def __init__(self, accountHolder):
        # BankAccount methods can access self._balance, but code outside of
        # this class should not:
        ❶ self._balance = 0
        ❷ self._name = accountHolder
        with open(self._name + 'Ledger.txt', 'w') as ledgerFile:
            ledgerFile.write('Balance is 0\n')

    def deposit(self, amount):
        ❸ if amount <= 0:
            return # Don't allow negative "deposits".
        self._balance += amount
        ❹ with open(self._name + 'Ledger.txt', 'a') as ledgerFile:
            ledgerFile.write('Deposit ' + str(amount) + '\n')
            ledgerFile.write('Balance is ' + str(self._balance) + '\n')

    def withdraw(self, amount):
        ❺ if self._balance < amount or amount < 0:
            return # Not enough in account, or withdraw is negative.
        self._balance -= amount
        ❻ with open(self._name + 'Ledger.txt', 'a') as ledgerFile:
            ledgerFile.write('Withdraw ' + str(amount) + '\n')
            ledgerFile.write('Balance is ' + str(self._balance) + '\n')
```